

Análisis léxico con Flex

Márquez Corona Danna Lizette

- Determinar los conjuntos N , Σ y el símbolo inicial S . (0.5 pts.)
 - N : declaraciones, declaracion, tipo, lista-var, sentencias, sentencia, expresion
 - Σ : int, float, identificador, numero, =, +, -, *, /, :, ,, if, else, while, (,)
 - S : programa
- Mostrar en el archivo el proceso de eliminación de ambigüedad o justificar, en caso de no ser necesario. (1 pts.).

Puesto que ejemplos como $a+b*c$ nos puede dar diferentes resultados, tenemos que adecuar las reglas para que nos de un unico resultado, esto se puede hacer de la siguiente forma:

```
expresion : expresion_suma
expresion_suma : expresion_suma + expresion_mult — expresion_suma
                -expresion_mult — expresion_mult
expresion_mult : expresion_mult * expresion_base — expresion_mult /
                expresion_base — expresion_base
expresion_base : identificador — numero — ( expresion )
```

Así podemos derivar $a + (b * c)$:

```
expresion : expresion_suma
: expresion_suma + expresion_mult
: expresion_mult + expresion_mult
: expresion_base + expresion_mult
: a + (b * c)
```
- Mostrar en el archivo el proceso de eliminación de la recursividad izquierda o justificar, en caso de no ser necesario. (1 pts.)

Podemos notar que sí hay recursividad por la izquierda, por lo tanto, para eliminarla usamos la tecnica de eliminación de recursividad agregando una nueva regla con un simbolo terminal a las reglas que necesiten eliminar la recursividad:

declaraciones : declaracion declaraciones'
 declaraciones' : declaracion declaraciones' $\rightarrow \epsilon$
 lista_var : identificador lista_var'
 lista_var' : , identificador lista_var' $\rightarrow \epsilon$
 sentencias : sentencia sentencias'
 sentencias' : sentencia sentencias' $\rightarrow \epsilon$

- Mostrar en el archivo el proceso de factorización izquierda o justificar, en caso de no ser necesario. (1 pts.)
 No es necesaria ya que después de eliminar recursividad izquierda y ambigüedad, las producciones se han reescrito de forma que cada conjunto de reglas para un mismo no terminal comienza con símbolos diferentes o se ha desglosado en componentes separados.

- Mostrar en el archivo los nuevos conjuntos N y P. (0.5 pts.)
 N: programa, declaraciones, declaraciones', declaracion, tipo, lista_var, lista_var', sentencias, sentencias', sentencia, sentencia_matched, sentencia_unmatched, expresion, expresion_suma, expresion_mult, expresion_base
 P:
 programa $\rightarrow$ declaraciones sentencias
 declaraciones $\rightarrow$ declaracion declaraciones'
 declaraciones' $\rightarrow$ declaracion declaraciones' | ϵ
 declaracion $\rightarrow$ tipo lista_var ;
 tipo $\rightarrow$ int | float
 lista_var $\rightarrow$ identificador lista_var'
 lista_var' $\rightarrow$, identificador lista_var' | ϵ
 sentencias $\rightarrow$ sentencia sentencias'
 sentencias' $\rightarrow$ sentencia sentencias' | ϵ
 sentencia $\rightarrow$ sentencia_matched | sentencia_unmatched
 sentencia_matched $\rightarrow$ identificador = expresion ;
 | if (expresion) sentencia_matched else sentencia_matched
 | while (expresion) sentencia_matched
 sentencia_unmatched $\rightarrow$ if (expresion) sentencia
 | if (expresion) sentencia_matched else sentencia_unmatched
 | while (expresion) sentencia
 expresion $\rightarrow$ expresion_suma
 expresion_suma $\rightarrow$ expresion_suma + expresion_mult

| expresion_suma - expresion_mult
 | expresion_mult
 expresion_mult $\rightarrow$ expresion_mult * expresion_base
 | expresion_mult / expresion_base
 | expresion_base
 expresion_base $\rightarrow$ identificador
 | numero
 | (expresion)